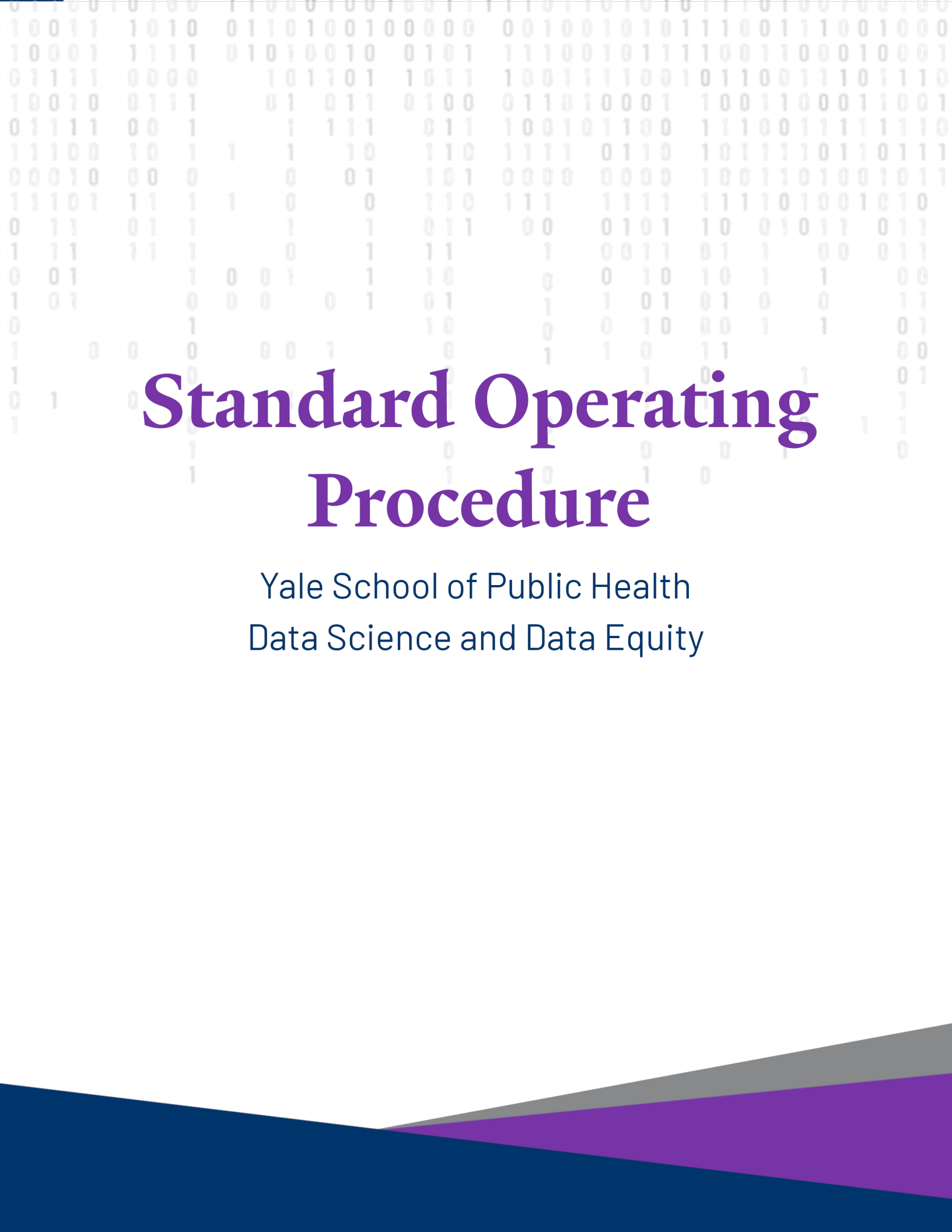


The background features a light gray grid of binary code (0s and 1s) that fades out towards the right. At the bottom of the slide, there are two overlapping geometric shapes: a dark blue triangle on the left and a purple triangle on the right, both pointing upwards.

Standard Operating Procedure

Yale School of Public Health
Data Science and Data Equity

The purpose of this document is to report how bare clone archivals are created and to provide a guideline for users to re-establish a working repository using them. This document is intended to be added to every archival GitHub repository where a bare clone copy is stored.

Developer: N/A

Authors:

- Shelby Golden, M.S.

Date Effective: June 3rd, 2026

Date Updated: June 3rd, 2026

Table of contents

How Bare Clone Archivals are Created	2
Continuity of Local-Only Files	3
Creating Bare Clone for Archival	3
Publishing an Archival Status Webpage	7
Archiving an Active Site in Place	7
Replacing a Site with a Bare Clone Archival Status Page	7
Reactivating a Bare Clone Archive	12
References	16

How Bare Clone Archivals are Created

Repository archivals are created if the codebases are no longer actively contributed to or maintained. This process applies to the following cases:

- **Inactive Repositories:** Repositories that are no longer active but are not being relocated.
- **Historical Copies:** Repositories retaining historical copies of a codebase that has since been relocated.

All archived repositories will be marked as such at the top of the project root directory `README.md` and in the repository Topic. The top of the `README.md` will also contain important details about the project, why it was archived, if it was moved, and important links as applicable. Some decommissioned codebases are retained in place or were moved to a consolidating repository storing similar archived projects.

Projects with an associated GitHub Pages or other kind of website had their existing pages modified or a standalone index page created at the original location, summarizing the archival status and the details outlined in the `README.md`. Some webpages may be temporarily marked for archival, with plans to remove the site altogether after sufficient time has passed to notify any potential users.

Examples

The following examples illustrate how the codebases for the PopHIVE project were archived after being moved to the PopHIVE GitHub Organization.

- Archived repository in the new location (not saved as a bare clone): [PopHIVE/data-gov](#)
- Consolidated archive of all related, decommissioned repositories that were moved (all saved as bare clones): [ysph-dsde/PopHIVE-Archival](#)

The following example illustrates how a project for Professor Ransome with an associated GitHub Pages website was archived after being relocated to his SOCAH Lab GitHub Organization.

- Active repository in the new location: [SOCAH-Lab/Church-Closures-Dashboard](#)
- Archived repository in the old location (saved as a bare clone with a stand alone index page): [ysph-dsde/church-closures/](#)

Continuity of Local-Only Files

Some repositories may contain local-only files that cannot be shared via GitHub, whether in a public or private repository. These are usually listed in the project's `.gitignore` file to ensure they are not Git-tracked.

Any local-only files were saved and stored in a location appropriate to their content security requirements and distributed among those who require access. For example, some may have been uploaded to a shared file system such as SharePoint. This method is also used to back up important files that are neither Git-tracked nor distributed through GitHub, such as non-public data, media, or excessively large files.

Reach out to the project's primary contact or admin.dsdeysph@yale.edu for additional support if one is not listed.

Creating Bare Clone for Archival

1. Create a bare clone of the repository.

Command-Line Interface

```
cd "~/FILE-LOCATION/"
git clone --bare git@github.com:EXAMPLE-USER/ORIGINAL-REPOSITORY.git
```

2. Repositories' bare clone files may contain large `*.pack` files that prevent uploading to a GitHub repository. To resolve this issue, the files can be unpacked and repacked with an imposed size limit.

Command-Line Interface

```
# Open directory with the pack object
cd "ORIGINAL-REPOSITORY.git/objects/pack"

# Unpack the "pack-xxxx.pack" file, replacing "xxxx" with
# the file's actual digits
cat pack-xxxx.pack | git unpack-objects -q
```

```
# Repack the object with the file size limit imposed
git repack -a -d --max-pack-size=50m
```

3. **ALTERNATIVE:** If the global pack size setting is sufficiently small, specifying the size explicitly is not required.

Command-Line Interface

```
# Verify the current global setting
git config --global --get pack.packSizeLimit

# OPTIONAL: change to 50 MB if the limit is too large or not set
git config --global pack.packSizeLimit 50m

# Repack using the global setting
git repack -a -d -f
```

4. If you encounter any trouble repacking into smaller files, you can either repack using the following command or re-clone a fresh bare clone to restart the process.

Command-Line Interface

```
# Repack into one file without size limit
git repack -a -d -f --max-pack-size=999999999
```

5. **OPTIONAL:** This process will generate one *.pack, *.idx and *.rev file of the same name for each split version of the original pack file. In some cases, it will also generate a pack-xxxx.bitmap file.

Bitmaps accelerate Git object searches by providing an efficient way to store reachable information from a set of commits in a packfile or multi-pack index (MIDX). This is particularly useful for large repositories and can significantly speed up operations such as cloning and fetching¹⁻³.

A pack-xxxx.bitmap file is not required and can be deleted. If you would like to include

one but it does not generate automatically, you can manually add it using:

Command-Line Interface

```
# Repack using the global setting and add bitmap
git repack -a -d -f --write-bitmap-index
```

6. **IMPORTANT:** If the project has local-only files that were not shared through GitHub, and therefore not included in the bare clone, and need to be preserved, refer to [Continuity of Local-Only Files](#) for further directions.
7. If the intention is to store a bare clone of this repository in one consolidating similar projects, proceed with the next few steps. Otherwise, skip to **Step 11**.
8. Navigate to the local, GitHub-synced consolidating directory and upload the new bare clone.

Command-Line Interface

```
# Open the separate archival directory consolidating similar projects
cd "FILE-PATH/ARCHIVAL-REPO"

git add .
git commit -m "ARCHIVAL MESSAGE"
git push
```

9. Update the root directory `README.md` as needed and refresh the GitHub page to see the changes reflected.
10. **OPTIONAL:** The original GitHub repository can be deleted.
11. If the intention is to keep the bare clone in place, proceed with the remaining steps. Otherwise, return to **Step 7** to upload the bare clone to a consolidating repository.
12. To ensure no information is lost during this process, create a copy of the project directory elsewhere until the entire process is complete.

13. Synchronize your local repository with GitHub, then move the bare clone into the project's root directory.
14. Delete all files, including hidden ones, except for the following:

/ORIGINAL-REPOSITORY/

```
.gitignore
ORIGINAL-REPOSITORY.git
README.md
```

15. Ensure all work is saved and committed.

Command-Line Interface

```
# Re-initialize Git
git init

# Commit changes
git add .
git commit -m "ARCHIVAL MESSAGE"

# Re-add the GitHub repository reference
git remote add origin git@github.com:EXAMPLE-USER/NEW-REPOSITORY.git #
↪ SSH
git remote add origin https://github.com/EXAMPLE-USER/NEW-REPOSITORY.git
↪ # HTTPS

# Forcefully update
git push origin main --force
```

16. Upload **ADD THE SHORTENED DIRECTIONS HERE**
17. Refresh the GitHub page to confirm the changes are reflected correctly, then delete the temporarily copied project directory.

Publishing an Archival Status Webpage

Repositories with an associated GitHub Pages or other kind of website will also need their archival status reported through the same URL of the old site. This allows users who return to the website to see that it has been decommissioned, is no longer being contributed to, or has been relocated.

In addition to the steps outlined previously, and depending on which are applicable to the situation, the site can either remain up as-is with a small note at the top of pages, or the entire site can be archived as a bare clone with a standalone index page published in its place indicating the change. Unless the site is intended to be continually referenced despite no longer being updated, the bare clone option is preferable.

Archiving an Active Site in Place

On the top of all pages, add a header or banner at the top of the page indicating something along the lines of:

Top of Each Webpage

! This Site Has Been Archived

This website has been archived and is no longer actively maintained or contributed to. You are welcome to continue using it as it was at the time of its last release; however, please be aware that some information, resources, or links may be outdated, broken, or no longer relevant.

Replacing a Site with a Bare Clone Archival Status Page

1. Update the root directory `README.md` with the relevant archival details outlined in Document the Archival Details, then commit this and any other outstanding changes.
2. To ensure no information is lost during this process, create a copy of the project directory elsewhere until the entire process is complete.

3. Synchronize your local repository with GitHub and then create a bare clone within the project's local directory.

Command-Line Interface

```
cd "~/FILE-LOCATION/PROJECT-NAME"

git pull
git clone --bare git@github.com:EXAMPLE-USER/ORIGINAL-REPOSITORY.git
```

4. Repositories' bare clone files may contain large *.pack files that prevent uploading to a GitHub repository. To resolve this issue, the files can be unpacked and repacked with an imposed size limit.

Command-Line Interface

```
# Open directory with the pack object
cd "ORIGINAL-REPOSITORY.git/objects/pack"

# Unpack the "pack-xxxx.pack" file, replacing "xxxx" with
# the file's actual digits
cat pack-xxxx.pack | git unpack-objects -q

# Repack the object with the file size limit imposed
git repack -a -d --max-pack-size=50m
```

5. **ALTERNATIVE:** If the global pack size setting is sufficiently small, specifying the size explicitly is not required.

Command-Line Interface

```
# Verify the current global setting
git config --global --get pack.packSizeLimit

# OPTIONAL: change to 50 MB if the limit is too large or not set
git config --global pack.packSizeLimit 50m
```

```
# Repack using the global setting
git repack -a -d -f
```

6. If you encounter any trouble repacking into smaller files, you can either repack using the following command or re-clone a fresh bare clone to restart the process.

Command-Line Interface

```
# Repack into one file without size limit
git repack -a -d -f --max-pack-size=999999999
```

7. **OPTIONAL:** This process will generate one *.pack, *.idx and *.rev file of the same name for each split version of the original pack file. In some cases, it will also generate a pack-xxxx.bitmap file.

Bitmaps accelerate Git object searches by providing an efficient way to store reachable information from a set of commits in a packfile or multi-pack index (MIDX). This is particularly useful for large repositories and can significantly speed up operations such as cloning and fetching¹⁻³.

A pack-xxxx.bitmap file is not required and can be deleted. If you would like to include one but it does not generate automatically, you can manually add it using:

Command-Line Interface

```
# Repack using the global setting and add bitmap
git repack -a -d -f --write-bitmap-index
```

8. **IMPORTANT:** If the project has local-only files that were not shared through GitHub, and therefore not included in the bare clone, and need to be preserved, refer to [Continuity of Local-Only Files](#) for further directions.

⚠️ Modify Steps as Needed for Your Setup

The following steps assume that the project is a GitHub Pages website generated using Quarto in RStudio. If it is not, adapt the steps as applicable to the languages and website type used.

9. Delete all files, including hidden ones, except for the following:

/ORIGINAL-REPOSITORY/

```
_quarto.yml/
.gitignore
ORIGINAL-REPOSITORY.git
ORIGINAL-REPOSITORY.Rproj
index.qmd
README.md
```

10. Launch the project by opening the *.Rproj file in RStudio.
11. Open *_quarto.yml and index.qmd in RStudio.
12. Navigate back to the project root directory and run a local preview of the website. It will open in a browser at the "localhost:[port]/web/" address.

Command-Line Interface

```
cd "../.." # Return to the root directory
quarto preview # Generate a preview of the site
```

13. In *_quarto.yml, delete all main menu links under navbar: and save. If there are any scripts, links, or styling references that have been deleted but still interfere with the display of the preview site, remove them from *_quarto.yml as well.
14. In the index.qmd file, copy the README.md contents and organize them into an "Overview" section and an "Archival Details" section. Additionally, add the following notice at the top of the page.

```
index.qmd
```

```
:::.callout-important
## This Site Has Been Archived and Moved
:::
```

15. Ensure all work is saved and committed.

Command-Line Interface

```
# Re-initialize Git
git init

# Commit changes
git add .
git commit -m "ARCHIVAL MESSAGE"

# Re-add the GitHub repository reference
git remote add origin git@github.com:EXAMPLE-USER/NEW-REPOSITORY.git #
↪ SSH
git remote add origin https://github.com/EXAMPLE-USER/NEW-REPOSITORY.git
↪ # HTTPS

# Forcefully update
git push origin main --force
```

16. Delete the gh-pages branch in GitHub, accessible under “View all branches”.
17. Publish the standalone archival webpage. Follow the prompts, entering “Yes” to proceed with updating the site and any required passwords when prompted.

Command-Line Interface

```
quarto publish gh-pages
```

18. Upload **ADD THE SHORTENED DIRECTIONS HERE**
19. Refresh the GitHub page to confirm the changes are reflected correctly, then delete the temporarily copied project directory.

Reactivating a Bare Clone Archive

If you are looking to re-establish the project, use the following directions and refer to [Continuity of Local-Only Files](#) for additional details pertaining to project-relevant content not stored in the bare clone.

1. [Log in](#) to your personal GitHub account.
2. In the top-right of the page navigation bar, select the **Create new...** dropdown menu and click **New repository**.
3. Fill out the following sections:
 - a. Adjust the GitHub account owner as needed and create the name for the new repository.
 - b. It is good practice to initially set the repository to "Private".
 - c. Do **NOT** use a template or include a description, README.md, .gitignore, or license.
4. Define the project rulesets under **Settings** → **Rules**. Refer to Repository Rulesets for additional details.
5. Download the *.git bare clone file for the repository you want to restore.
6. Move the file into the directory where you want to save the project on your local device.
7. In your command-line application, navigate into the bare clone directory.

Command-Line Interface

```
cd "~/FILE-LOCATION/ORIGINAL-REPOSITORY.git"
```

8. To link the bare codebase to an empty GitHub repository, the repository location must be defined using either its HTTPS or SSH URL. The choice depends on individual preference and system configuration, and each collaborator may use a different protocol when connecting to the same repository.

In the newly created GitHub repository, under **Quick setup**, you will find the repository's SSH or HTTPS URL. Copy one of these URLs to define the remote location and transfer protocol you want to use between your local device and this GitHub repository.

For example, if the repository name is "NEW-REPOSITORY," the URLs will look like this:

```
# SSH
git@github.com:EXAMPLE-USER/NEW-REPOSITORY.git

# HTTPS
https://github.com/EXAMPLE-USER/NEW-REPOSITORY.git
```

9. Designate the remote location and transfer protocol (SSH or HTTPS), then push the project as a mirror to the remote repository. This will also extract the project contents.

Command-Line Interface

```
# SSH
git push --mirror git@github.com:EXAMPLE-USER/NEW-REPOSITORY.git

# HTTPS
git push --mirror https://github.com/EXAMPLE-USER/NEW-REPOSITORY.git
```

10. Refresh the GitHub page to see the changes reflected.
11. Delete the bare cloned file used to create a new remote repository.

Command-Line Interface

```
cd .. # Go back one file location
rm -rf git@github.com:EXAMPLE-USER/ORIGINAL-REPOSITORY.git
```

12. After the project contents have been extracted and pushed to GitHub, they can be cloned to establish a usable local copy.

Command-Line Interface

```
cd "~/FILE-LOCATION/"

git clone git@github.com:EXAMPLE-USER/NEW-REPOSITORY.git # SSH
git clone https://github.com/EXAMPLE-USER/NEW-REPOSITORY.git # HTTPS
```

13. Version control via Git is required for all codebase projects. Confirm that Git has been initialized by checking the project status.

If Git has already been initialized, it will appear as active and tracking files. If it has not, and you get the following error, proceed with the next steps.

Command-Line Interface

```
git status # Show current status
```

Output

```
fatal: not a git repository (or any of the parent directories: .git)
```

14. If version control has not yet been initialized, set up Git in the project directory and create the primary branch. `main`.

Command-Line Interface

```
git init -b main
```

Output

```
Initialized empty Git repository in ~/PROJECT-NAME/.git/
```

15. **OPTIONAL:** Some projects may include or require a `.gitignore` file. Ensure that all intended exclusions are correctly specified in this file before committing and pushing the codebase to the new repository.
16. Running `git status` again should no longer produce an error, but will instead display any untracked files in the project directory, as shown in the example below. If this is the first commit following Git initialization, all project files will appear as untracked.

Command-Line Interface

```
git status    # Show current status
```

Output

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  README.md

nothing added to commit but untracked files present (use "git add" to
↪ track)
```

17. Confirm that any files intended to remain local or be Git-ignored are not listed among the untracked files above. When ready, stage and commit the changes.

References

1. git-scm. Pack and multi-pack bitmaps. <https://git-scm.com/docs/bitmap-format>.
2. git-scm. Glossary for version 2.39.0. <https://git-scm.com/docs/gitglossary/2.39.0>.
3. Konrad Scherer. Adventures with git and server packfile bitmaps. <https://kscherer.github.io/git/2015/05/15/git-and-bitmaps>.